

메타코딩의 자율주행 개발 일기

"AI 처음 배우는 사람도 이해하는 실무 스토리"

실무에서 겪은 문제와 해결 과정을 따라가다 보면 어느새 강화학습을 이해하게 됩니다!

목차

- 프롤로그: 갑작스러운 프로젝트
- 1화: 첫 성공의 함정 (v1)
- 2화: 문제의 원인을 찾아서 (v2)
- 3화: 진짜 실력을 증명하라 (v3)
- 4화: 더 빠르게, 더 똑똑하게 (v4)
- 5화: 현장의 예상치 못한 변수 (v5)
- 에필로그: 배운 것들

프롤로그: 갑작스러운 프로젝트

 17 어느 월요일 아침

"메타코딩 씨, 잠깐 와봐요."

팀장님의 부름을 받고 회의실에 들어갔습니다. 테이블에는 작은 로봇 청소기가 놓여 있었습니다.

"이거 보이죠? 우리 회사에서 자율주행 청소 로봇을 만들기로 했어요. 메타코딩 씨가 AI를 담당해주세요."

"네? 저요? 저 강화학습은 책으로만 봤는데요..."

"걱정 마세요. 간단한 시뮬레이터부터 시작하면 돼요. 2주 안에 데모 보여주시면 됩니다!"

 메타코딩의 생각

'2주? 강화학습? AI?'

집에 가서 인터넷을 뒤져봤습니다. DQN이라는 게 있다고 합니다. "게임 AI 만드는데 쓴다던데... 청소 로봇도 비슷하지 않을까?"

 일단 시작하자!

다음날부터 열심히 코드를 짭니다.

목표:

- 시작점(노란색)에서 출발
- 목적지(초록색)까지 도달
- 벽(검은색)에 부딪히면 안 됨

간단해 보였습니다.

AI는 무엇을 보고 움직이는가?

메타코딩이 코드를 짜기 전에, 먼저 이해해야 할 것이 있었습니다.

🤔 AI는 어떻게 길을 찾을까?

"잠깐, AI에게 뭘 알려줘야 하지?"

메타코딩은 고민에 빠졌습니다.

선택지 1: 전체 맵을 보여주기

"목적지는 (50, 50)이야. 전체 맵은 이렇게 생겼어."
→ 너무 많은 정보! (맵이 바뀌면?)

선택지 2: 나침반처럼 방향만 알려주기 (선택!)

"목적지가 저쪽 방향이야. 주변에 벽이 있네."
→ 간단하고 일반화 가능!

📊 AI가 받는 정보 (상태 벡터)

메타코딩은 AI에게 이런 정보만 주기로 했습니다:

```
상태 = [
    # 1. 주변 8칸 장애물 정보
    wall_up_left, wall_up, wall_up_right,
    wall_left, wall_right,
    wall_down_left, wall_down, wall_down_right,

    # 2. 현재 바라보는 방향
    direction, # 0=위, 1=오른쪽, 2=아래, 3=왼쪽

    # 3. 목적지가 어느 방향인지 (★ 핵심!)
    dx_to_goal, # "목적지가 X축으로 저쪽"
    dy_to_goal # "목적지가 Y축으로 저쪽"
]
```

중요한 점:

- AI는 목적지의 **절대 좌표**를 모름
- "맵의 (50, 50)에 목적지가 있다"가 아니라
- "**저쪽 방향으로 가면 된다**" 수준만 알

🎯 목적은 알고 있나요?

"그럼 AI는 어디로 가야 하는지 아는 건가요?"

네, 알고 있습니다! (보상으로)

```
if 목적지 도달:
    reward = +100 # "잘했어! 계속 이렇게 해!"
elif 목적지에 가까워짐:
    reward = +0.5 # "좋아, 제대로 가고 있어!"
elif 충돌:
    reward = -100 # "안 돼! 이걸 나빠!"
```

AI는 이 보상을 통해:

- "목적지로 가는 게 좋다"
- "충돌은 나쁘다"

를 배웁니다.

비유로 이해하기

❌ GPS (절대 좌표)
 "서울시 강남구 테헤란로 123번지로 가세요"
 → 주소가 바뀌면? 다시 프로그래밍!

✅ 나침반 (상대 방향)
 "목적지가 북동쪽입니다. 장애물 피하며 가세요"
 → 어디든 적용 가능! 일반화됨!

메타코딩은 이제 코드를 짤 준비가 되었습니다.

"좋아, 나침반처럼 방향만 알려주는 AI를 만들자!"

1화: 첫 성공의 함정 (v1)

simulator-v1 폴더 생성

메타코딩은 열심히 코드를 짭니다.

```
# config.py
NUM_EPISODES = 500 # 500번 학습하면 되겠지?
```

학습 시작!

```
cd simulator-v1
python train.py
```

화면에 작은 자동차가 나타났습니다.

처음 100번 (Episode 0-100):

 : "어디로 가지? 벽에 부딪쳐봤네. 이쪽은 아니구나."
(계속 벽에 충돌하면서 랜덤하게 움직임)

중간 200번 (Episode 100-300):

 : "아! 위로 가면 되는구나!"
(점점 목적지에 가까워짐)

마지막 200번 (Episode 300-500):

 : "완벽해! 매번 목적지에 도달!"
(성공률 80%!)

 성공이다!

"와! 되네? AI가 스스로 학습했어!"

메타코딩은 신났습니다. 모델을 저장했습니다.

```
python train.py
# 학습 완료! model_final.pth 저장됨
```

 데모 준비

"좋아, 이제 테스트해보자!"

```
python main.py
```

화면에 자동차가 나타나고, 매끄럽게 목적지에 도달했습니다.

"완벽해! 내일 팀장님께 보여드려야지!"

 밤늦게까지...

그런데 뭔가 이상했습니다.

자동차가 **항상 같은 경로**로만 움직였습니다.

시작점(노란색) → 위로만 쪽 → 오른쪽으로 쪽 → 목적지(초록색)

"뭐, 목적지에 도달하니까 괜찮겠지?"

💥 다음날, 팀장님 앞에서

"자, 보여드리겠습니다!"

메타코딩이 자신있게 프로그램을 실행했습니다.

그런데...

팀장님: "좋네요! 그런데 다른 맵에서도 작동하나요?"

메타코딩: "네? 다른 맵이요?"

팀장님이 설정을 바꿔서 다시 실행했습니다.

```
# 맵을 바꿔봤더니...
env = GridEnvironment(random_map=True)
```

결과:

🚗 : "어? 벽이 어디 있지?"
(벽에 계속 충돌!)
(전혀 목적지를 찾아가지 못함!)

팀장님: "음... 이 맵에서만 작동하는 건가요?"

메타코딩: "아... 그게..."

😞 대실패

회의실로 돌아왔습니다.

팀장님: "메타코딩 씨, 청소 로봇은 여러 집에서 작동해야 해요. 한 집에서만 작동하면 안 되죠."

메타코딩: "죄송합니다... 다시 해보겠습니다."

😞 무엇이 문제였을까?

집에 돌아와서 곰곰이 생각해봤습니다.

문제 1: 항상 같은 위치에서 출발

```
# car.py
def __init__(self, x, y):
```

```
self.direction = 0 # 항상 위쪽만 보고 시작!
```

"아! 항상 위쪽을 보고 시작하니까, 위로만 가는 법을 배운 거구나!"

문제 2: 항상 같은 맵에서 학습

```
# train.py
env = GridEnvironment() # 항상 같은 맵!
```

"한 집에서만 청소하는 법을 배웠네..."

문제 3: 너무 빨리 학습을 멈춤

```
# agent.py
self.epsilon_decay = 0.995 # 빨리 감소
```

"탐험을 너무 빨리 멈췄구나. 다양한 방법을 시도해보지 못했어."

☁ 깨달음

이게 바로 **과적합(Overfitting)**이구나!

비유:

학생이 수학 문제집의 1번 문제만 1000번 풀었어요.
1번 문제는 완벽하게 풀 수 있어요!

하지만 시험에는 2번 문제가 나왔어요.
→ 못 풀어요! ❌

이게 과적합입니다!

📝 메모장에 정리

메타코딩은 배운 것을 정리했습니다.

v1의 문제점:

- ❌ 항상 같은 방향에서 시작 → 한쪽으로만 이동
- ❌ 항상 같은 맵에서 학습 → 새로운 맵에서 실패
- ❌ 너무 빨리 탐험 중단 → 다양한 방법 못 배움

해결 방법:

- ✅ 여러 방향에서 시작해보자
- ✅ 여러 맵에서 학습하자

-  더 오래 탐험하자

"내일부터 다시 시작이다!"

2화: 문제의 원인을 찾아서 (v2)

 2주차 월요일

메타코딩은 v2 폴더를 만들고 새로 시작했습니다.

```
mkdir simulator-v2
cd simulator-v2
```

 해결책 1: 랜덤 방향으로 시작

Before (v1):

```
# car.py
def __init__(self, x, y):
    self.direction = 0 # 항상 위쪽
```

After (v2):

```
# car.py
import random

def __init__(self, x, y):
    self.direction = random.randint(0, 3) # 랜덤!
    # 0: 위, 1: 아래, 2: 왼쪽, 3: 오른쪽
```

효과:

```
Episode 1: 위쪽 보고 시작 → 위로 가는 법 배움
Episode 2: 왼쪽 보고 시작 → 왼쪽으로 가는 법 배움
Episode 3: 아래쪽 보고 시작 → 아래로 가는 법 배움
...

→ 모든 방향으로 가는 법을 배움! 
```

 해결책 2: 여러 맵에서 학습

Before (v1):

```
# train.py
env = GridEnvironment() # 항상 같은 맵
```

After (v2):

```
# config.py
NUM_MAPS = 20 # 20개 맵!
NUM_EPISODES = 3000 # 500 → 3000

# train.py
for episode in range(NUM_EPISODES):
    # 150 에피소드마다 맵 변경
    if episode % 150 == 0:
        map_id = random.randint(0, 19)
        env.reset_map(map_id)
```

효과:

맵 0 (빈 맵): 직진하는 법 배움
 맵 5 (중간 맵): 장애물 피하는 법 배움
 맵 10 (복잡한 맵): 미로 탈출하는 법 배움
 맵 15 (랜덤 맵): 처음 보는 맵 대처법 배움

→ 어떤 맵에서도 작동! 

해결책 3: 천천히 학습

Before (v1):

```
# agent.py
self.epsilon_decay = 0.995 # 빠르게 감소
```

After (v2):

```
# agent.py
self.epsilon_decay = 0.998 # 느리게 감소
```

무슨 뜻일까?

epsilon은 "탐험 확률"입니다.

```
epsilon = 1.0 (100%): "새로운 길을 시도해보자!" (탐험)
epsilon = 0.0 (0%): "아는 길로만 가자!" (활용)
```

v1의 문제:

Episode 1-50: epsilon = 1.0 → 0.6 (빠르게 감소)
 Episode 51: "이제 탐험 그만! 아는 길로만 가자!"

→ 한 가지 방법만 배움 ❌

v2의 개선:

Episode 1-500: epsilon = 1.0 → 0.6 (천천히 감소)
 Episode 501-1000: epsilon = 0.6 → 0.3 (계속 탐험!)
 Episode 1001-3000: epsilon = 0.3 → 0.01 (충분히 탐험 후 활용)

→ 다양한 방법을 배움! ✅

🔧 해결책 4: 새로운 길 찾기 보상

메타코딩은 생각했습니다.

"같은 길만 왔다갔다하면 안 되지 않을까?"

새로운 보상 시스템:

```
# car.py
class Car:
    def __init__(self, x, y):
        # ...
        self.visited_positions = set() # 방문한 곳 기록

    def move(self, action, environment):
        # 이동 후
        if (next_x, next_y) not in self.visited_positions:
            reward += 0.2 # 새로운 곳! 보너스!
            self.visited_positions.add((next_x, next_y))

        # 같은 곳만 반복하면
        if len(self.visited_positions) < self.steps * 0.5:
            reward -= 0.1 # 패널티!
```

효과:

🚗 : "아! 여기는 처음이네? (보너스 +0.2)"
 🚗 : "또 여기야... 같은 곳만 돌고 있네 (패널티 -0.1)"
 → 다양한 경로를 탐험! ✅

🔧 해결책 5: 학습 안정화

선배 개발자가 조언해줬습니다.

"Gradient Clipping이라는 걸 써봐요. 학습이 안정돼요."

```
# agent.py
def replay(self):
    # ... (기존 코드)
    loss.backward()

    # Gradient Clipping 추가!
    torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 1.0)

    self.optimizer.step()
```

무슨 뜻일까?

비유:

학생이 공부를 해요.

v1 (Gradient Clipping 없음):
 - 어떤 날은 10시간 공부 (너무 많아!)
 - 어떤 날은 0시간 공부 (너무 적어!)
 → 불안정! 😞

v2 (Gradient Clipping 있음):
 - 매일 2-3시간씩 꾸준히 공부
 → 안정적! 😊

🎮 다시 학습!

```
cd simulator-v2
python train.py
```

이번에는 시간이 더 걸렸습니다. (3000 에피소드니까요!)

Episode 0-500: 초보 단계

🚗 : "이 맵에서는 위로 가면 되고..."
 🚗 : "저 맵에서는 왼쪽으로 가면 되고..."
 (여러 맵에서 랜덤하게 탐험)

Episode 500-1500: 학습 시작

🚗 : "아하! 벽을 피하는 패턴이 있구나!"
 🚗 : "목적지 쪽으로 가면서 벽을 피하면 되겠다!"
 (패턴 발견!)

Episode 1500-3000: 마스터!

🚗 : "어떤 맵이든 벽을 피하고 목적지에 도달!"
 (성공률 70%!)

🎯 테스트!

메타코딩은 떨리는 마음으로 테스트를 실행했습니다.

```
python main.py
```

결과:

맵 A (단순): ✅ 성공!
 맵 B (복잡): ✅ 성공!
 맵 C (랜덤): ✅ 성공!
 완전히 새로운 맵: ✅ 성공!

"와! 이번엔 진짜 되는데?"

👤 팀장님께 보고

다음날, 자신감 있게 팀장님께 보여드렸습니다.

팀장님: "오! 이번엔 다른 맵에서도 잘 작동하네요!"

팀장님이 여러 맵으로 바뀌가며 테스트했습니다.

테스트 1: 빈 맵 → ✅ 성공
 테스트 2: 복잡한 맵 → ✅ 성공
 테스트 3: 완전 랜덤 맵 → ✅ 성공

팀장님: "훨씬 좋아졌어요! 그런데..."

메타코딩: "네?"

팀장님: "이게 정말 새로운 맵에서도 작동하는 건지 확실한가요? 혹시 20개 맵을 외운 건 아니죠?"

메타코딩: "아... 그건..."

😓 또 다른 문제?

팀장님 말이 맞았습니다.

현재 상황:

학습: 맵 0-19에서 학습 (20개)
테스트: 맵 0-19에서 테스트 (같은 맵!)

문제: 정말 새로운 맵인지 확실하지 않음!

비유:

학생이 문제집 1-20번을 풀어봤어요.
시험에 1-20번이 나왔어요.
100점 받았어요!

→ 진짜 실력일까요? 아니면 문제를 외운 걸까요? 😓

 메모장에 추가

v2에서 해결한 것:

-  랜덤 방향 시작
-  여러 맵에서 학습
-  느린 탐험 감소
-  경로 다양성 보상
-  학습 안정화

새로운 문제:

-  학습한 맵에서만 테스트 → 진짜 실력인지 모름

"테스트를 다르게 해야겠구나!"

3화: 진짜 실력을 증명하라 (v3)

 3주차

선배 개발자를 찾아갔습니다.

메타코딩: "선배님, 제 AI가 정말 똑똑한 건지 확인하려면 어떻게 해야 해요?"

선배: "훈련셋과 테스트셋을 분리해야죠!"

메타코딩: "네? 그게 뭔데요?"

훈련셋 vs 테스트셋

선배가 설명해줬습니다.

비유:

학생이 공부를 해요.

연습 문제집 (훈련셋):

- 1-20번 문제를 풀어봄
- 이걸로 공부함

시험 문제 (테스트셋):

- 21-30번 문제 (처음 보는 문제!)
- 이걸로 실력을 평가함

→ 진짜 실력을 알 수 있음!

현재 문제:

v2:

- 학습: 맵 0-19
- 테스트: 맵 0-19 (같은 맵!)

→ 시험에 연습문제가 그대로 나온 것과 같음!

→ 진짜 실력인지 알 수 없음!

해결 방법:

v3:

- 학습: 맵 0-19
- 테스트: 완전히 새로운 랜덤 맵!

→ 시험에 새로운 문제가 나옴!

→ 진짜 실력을 알 수 있음!

테스트 코드 수정

메타코딩은 `simulator-v3` 폴더를 만들고, v2 코드를 복사한 후 수정했습니다.

train.py (학습):

```
# 학습은 v2와 동일
for episode in range(NUM_EPISODES):
    if episode % 150 == 0:
        map_id = random.randint(0, 19) # 0-19번 맵
        env.reset_map(map_id)
    # ... 학습
```

main.py (테스트) - 수정!

```
# Before (v2)
env = GridEnvironment() # 학습한 맵 중 하나

# After (v3)
env = GridEnvironment(random_map=True) # 완전히 새로운 맵!
```

main.py에 경고 메시지 추가:

```
print("⚠ 중요: 훈련에 사용하지 않은 새로운 맵에서 테스트합니다!")
print("    → 과적합 여부를 확인하고 일반화 능력을 검증합니다.")
```

🎮 학습 및 테스트

학습은 v2와 똑같이 진행되었습니다.

```
cd simulator-v3
python train.py
# ... 3000 에피소드 학습 완료!
```

이제 테스트입니다!

```
python main.py
```

화면 메시지:

```
⚠ 중요: 훈련에 사용하지 않은 새로운 맵에서 테스트합니다!
    → 과적합 여부를 확인하고 일반화 능력을 검증합니다.
```

```
🎮 테스트 시작! (일반화 테스트)
맵 ID: NEW_RANDOM_MAP_1
```

결과:

테스트 1: 완전히 새로운 랜덤 맵 →  성공!
 테스트 2: 또 다른 랜덤 맵 →  성공!
 테스트 3: 복잡한 새 맵 →  성공!

Success Rate: 68%

"오! 처음 보는 맵에서도 작동한다!"

 깨달음

메타코딩의 메모:

이제야 알겠다!

v2:

- 같은 맵으로 테스트 → 90% 성공
- 새로운 맵으로 테스트 → 68% 성공

90%는 과장된 수치였구나!
 진짜 실력은 68%였어!

하지만 68%도 나쁘지 않아!
 적어도 이제 "진짜 실력"을 알게 됐어!

** 팀장님께 보고 (3번째)**

자신감 있게 보고했습니다.

메타코딩: "이번에는 완전히 새로운 맵에서 테스트했습니다!"

팀장님이 여러 번 테스트를 돌려봤습니다.

팀장님: "오! 처음 보는 맵에서도 잘 작동하네요. 이제야 믿을 수 있겠어요!"

메타코딩: "감사합니다!"

팀장님: "그런데 학습 속도가 좀 느린 것 같아요. 3000번이나 해야 하니까... 더 빨리 학습할 방법은 없을까요?"

메타코딩: "아... 네, 찾아보겠습니다."

 정리

v3에서 배운 것:

-  훈련셋과 테스트셋을 분리해야 진짜 실력을 알 수 있다
-  같은 데이터로 학습하고 테스트하면 과대평가된다
-  68% 성공률 = 진짜 실력

새로운 과제:

- ? 학습 속도 개선 필요
- ? 3000 에피소드는 너무 오래 걸림

"학습을 더 빠르게 할 방법을 찾아보자!"

4화: 더 빠르게, 더 똑똑하게 (v4)**17 4주차**

메타코딩은 온라인 강의를 보다가 흥미로운 개념을 발견했습니다.

"Learning Rate Scheduling"

🤔 Learning Rate가 뭐지?

비유:

학생이 공부할 때 교재를 읽는 속도예요.

빠르게 읽기 (큰 Learning Rate):

- 장점: 전체 내용을 빠르게 훑을 수 있음
- 단점: 세부 내용을 놓칠 수 있음

천천히 읽기 (작은 Learning Rate):

- 장점: 세부 내용까지 꼼꼼하게 읽을 수 있음
- 단점: 시간이 오래 걸림

💡 아이디어!

"처음에는 빠르게 읽고, 나중에는 천천히 읽으면 어떨까?"

Learning Rate Scheduling:

초반 (Episode 0-1000):

- 큰 Learning Rate (0.001)
- 빠르게 전체 패턴 파악!

중반 (Episode 1000-2000):

- 중간 Learning Rate (0.0005)
- 안정적으로 학습!

후반 (Episode 2000-3000):

- 작은 Learning Rate (0.0001)
- 세밀하게 조정!

효과:

v3 (고정된 Learning Rate):

- 처음부터 끝까지 같은 속도
- 느리지만 안정적

v4 (동적 Learning Rate):

- 처음에는 빠르게, 나중에는 천천히
- 빠르면서도 정확!

코드 수정

`simulator-v4` 폴더를 만들고 수정했습니다.

agent.py 수정:

```
class DQNAgent:
    def __init__(self):
        # ... (기존 코드)

        # Learning Rate Scheduling 추가!
        self.initial_lr = 0.001 # 초기 학습률 (큰 값)
        self.min_lr = 0.0001 # 최소 학습률 (작은 값)
        self.current_lr = self.initial_lr

    def update_learning_rate(self, episode):
        """에피소드마다 학습률 감소"""
        decay_rate = 0.998
        self.current_lr = self.initial_lr * (decay_rate ** episode)
        self.current_lr = max(self.current_lr, self.min_lr)

        # 옵티마이저 업데이트
        for param_group in self.optimizer.param_groups:
            param_group['lr'] = self.current_lr
```

train.py 수정:

```
for episode in range(NUM_EPISODES):
    # Learning Rate 업데이트 추가!
    agent.update_learning_rate(episode)

    # ... (나머지는 v3와 동일)

    # 로그에 Learning Rate 표시
    if (episode + 1) % 100 == 0:
        print(f"Episode {episode+1} | "
              f"Score: {avg_score:.2f} | "
              f"LR: {agent.current_lr:.6f}") # 추가!
```

 학습 시작!

```
cd simulator-v4
python train.py
```

화면 출력:

```
Episode 100 | Score: 1.2 | LR: 0.000820
Episode 500 | Score: 3.8 | LR: 0.000368
Episode 1000 | Score: 5.2 | LR: 0.000135
Episode 2000 | Score: 6.1 | LR: 0.000100
Episode 3000 | Score: 6.8 | LR: 0.000100
```

"오! Learning Rate가 점점 줄어드네!"

 결과 비교

메타코딩은 v3와 v4를 비교해봤습니다.

v3 (고정 Learning Rate):

```
Episode 500: 평균 점수 3.2
Episode 1000: 평균 점수 4.8
Episode 2000: 평균 점수 5.8
Episode 3000: 평균 점수 6.5
```

최종 성공률: 68%

v4 (동적 Learning Rate):

```
Episode 500: 평균 점수 3.8 (더 높음!)
Episode 1000: 평균 점수 5.2 (더 높음!)
Episode 2000: 평균 점수 6.1 (비슷)
Episode 3000: 평균 점수 6.8 (더 높음!)
```

최종 성공률: 73% (더 높음!)

시간 비교:

```
v3: Episode 2000에서 성능 6.0 도달
v4: Episode 1500에서 성능 6.0 도달
```

→ 약 25% 빠름! 

💡 왜 빠를까?

메타코딩의 이해:

v3 (고정):

🚗 : "천천히... 조심조심..."
(처음부터 끝까지 같은 속도)

v4 (동적):

🚗 : "처음엔 빠르게 대충 파악! 🏃 "
(Episode 0-1000: 큰 Learning Rate)

🚗 : "이제 안정적으로 학습! 🧑 "
(Episode 1000-2000: 중간 Learning Rate)

🚗 : "마지막은 세밀하게 조정! 🐌 "
(Episode 2000-3000: 작은 Learning Rate)

→ 빠르고 정확!

👤 팀장님께 보고 (4번째)

메타코딩: "이번엔 학습 속도를 25% 개선했습니다!"

팀장님: "오! 어떻게 했어요?"

메타코딩: "Learning Rate Scheduling을 적용했습니다. 처음엔 빠르게, 나중엔 천천히 학습하는 거예요."

팀장님: "좋아요! 그런데..."

메타코딩: "또 뭔가 있나요?" 🤔

팀장님: "실제 제품은 같은 공간에서 반복해서 작동하잖아요. 매번 신경망을 계산하는 게 배터리 낭비 아닐까요?"

메타코딩: "아... 추론(Inference) 속도 최적화요?"

팀장님: "네, 같은 환경에서는 더 빠르게 실행할 방법이 있을까요?"

메타코딩: "음... 생각해보겠습니다!"

📝 정리

v4에서 배운 것:

- ✅ Learning Rate Scheduling으로 학습 속도 25% 향상
- ✅ 처음엔 빠르게, 나중엔 천천히
- ✅ 더 빠르고 더 정확한 학습

새로운 문제:

- ? 동적으로 변하는 환경
- ? 예상치 못한 장애물

"이제 진짜 현장 상황을 시뮬레이션해봐야겠구나!"

5화: 추론 속도를 더 빠르게! (v5)

5주차

메타코딩은 v4를 완성하고 뿌듯해했습니다.

"좋아! 학습도 빠르고, 성능도 좋고... 이제 끝난 건가?"

선배의 조언

선배 개발자를 만났습니다.

선배: "메타코딩 씨, 학습은 한 번만 하면 되잖아요? 그런데 추론(Inference)은 매번 해야 하죠."

메타코딩: "네? 무슨 말씀이세요?"

선배: "같은 맵에서 계속 실행할 때를 생각해봐요. 매번 Policy Network를 계산하는 게 비효율적이지 않을까요?"

아이디어: 캐싱!

메타코딩은 생각했습니다.

"그리고 보니... 같은 맵에서 계속 실행하면 비슷한 경로로 가는데, 매번 신경망 계산(45ms)을 하는 게 낭비 아닐까?"

성공한 경로를 캐싱해두면 빠르지 않을까?"

비유:

내비게이션이:

- 매번 경로 계산 (느림) ❌

vs

- 자주 가는 길은 저장 (빠름) ✅

캐싱 시스템 구현

`simulator-v5` 폴더를 만들었습니다.

agent.py에 ActionCache 추가:

```
class ActionCache:
    """성공한 경험을 캐싱하는 시스템"""
    def __init__(self):
        self.cache = {} # 상태 → 행동 매핑

    def get(self, state):
        """캐시에서 행동 가져오기"""
        state_key = tuple(round(x, 2) for x in state)
        if state_key in self.cache:
            return self.cache[state_key]['action']
        return None
```

```
def store(self, state, action, success):
    """성공한 행동만 저장"""
    if success:
        state_key = tuple(round(x, 2) for x in state)
        self.cache[state_key] = {'action': action}
```

config.py 설정:

```
# v5 신기능: 캐싱 시스템
USE_CACHE = True # 캐시 활성화!
```

🎮 첫 번째 테스트

```
cd simulator-v5
# 학습은 v4와 동일하게
python train.py

# 테스트는 캐시 활성화!
python main.py
```

같은 맵에서 반복 실행:

```
Step 1-10: Policy 계산 (45ms), 캐시 비어있음
Step 11-20: Policy 계산 + 캐시 쌓이기 시작
Step 21-50: 대부분 캐시 히트! (예상: 12ms로 빨라질 것)
```

결과:

음... 속도가 안 빨라지네? 🤔

메타코딩: "어? 캐시를 사용하는데 왜 느리지?"

🤔 선배의 설명

선배를 찾아갔습니다.

메타코딩: "선배님, 캐시를 만들었는데 속도가 안 빨라져요!"

선배: "아, 그건 의도한 거예요. 현재 v5는 **교육용** 코드거든요."

메타코딩: "네? 교육용이요?"

💡 두 가지 전략: 정적 vs 동적

선배가 설명해줬습니다.

"실무에서는 환경에 따라 두 가지 전략이 필요해요."

전략 1: 정적 환경 (장애물 고정)

```
# 공장, 창고 등 고정된 환경
def select_action(state):
    # Cache를 먼저 확인! (빠름)
    if cache.has(state):
        return cache.get(state) # 12ms ✅

    # Cache에 없으면 Policy 계산
    return policy_net(state).argmax() # 45ms
```

효과:

- 캐시 히트: 12ms (3.75배 빠름!)
- 빠른 실행

전략 2: 동적 환경 (장애물 변화)

```
# 사람 많은 곳, 변화하는 환경
def select_action(state):
    # 항상 Policy 먼저! (안전)
    policy_action = policy_net(state).argmax() # 45ms
    return policy_action # ✅

    # Cache는 참고만 (또는 무시)
```

효과:

- 항상 최신 상태 반영
- 안전한 실행

메타코딩: "아하! 그럼 v5는 두 전략을 모두 보여주는 교육용 코드구나!"

선배: "맞아요! 현재 v5는 항상 Policy를 계산해서 안전성을 보장하고, Cache는 Policy와 일치하는지 확인만 해요."

🔧 Policy > Cache 원칙 구현

선배: "현재 v5 코드는 교육용이에요. 이해를 돕기 위해 항상 Policy를 먼저 계산하죠."

현재 v5 코드 (교육용):

```
class DQNAgent:
    def select_action(self, state, training=False):
        # 1 Policy가 먼저 판단! (항상 실행!)
        policy_action = self.policy_net(state).argmax() # 45ms
```

```

# 2 캐시 확인 (참고만)
if self.use_cache:
    cached_action = self.cache.get(state) # 1ms

    if cached_action is not None:
        if cached_action == policy_action:
            # ✅ 일치: 안전
            return policy_action
        else:
            # ⚠ 불일치: Policy 우선! (안전)
            return policy_action

# 3 최종 결정은 항상 Policy
return policy_action

```

결과:

- 항상 45ms 소요 (캐시 사용해도!)
- 왜? Policy를 항상 먼저 계산하니까!

메타코딩: "그럼 실전에서는 어떻게 하나요?"

선배: "환경에 따라 다르게 구현해요!"

💡 비유로 이해하기

선배가 설명해줬습니다.

비유: 운전자와 네비게이션

Policy Network = 운전자의 눈 👁

- 실시간으로 도로 상황 확인
- 갑자기 나타난 공사 현장 감지
- 최종 판단

Action Cache = 네비게이션 📱

- 평소 다니던 길 기억
- 빠른 경로 제안
- 참고 자료일 뿐

Policy > Cache:

네비: "직진하세요"

운전자: "어? 앞에 공사중이네? 우회해야겠다"

→ 운전자(Policy) 판단 우선! ✅

🔪 캐싱의 한계 테스트 (동적 장애물)

메타코딩: "그럼 캐싱의 한계를 직접 보고 싶은데요?"

선배: "좋은 생각이에요! 동적 장애물 시스템으로 테스트해봅시다."

동적 장애물 시스템 추가

config.py 설정:

```
# 캐싱 한계 테스트용
ENABLE_DYNAMIC_OBSTACLES = True # 동적 장애물 활성화
OBSTACLE_SPAWN_INTERVAL = 50 # 50스텝마다 생성
OBSTACLE_LIFETIME = 30 # 30스텝 후 사라짐
```

environment.py에 추가:

```
class GridEnvironment:
    def update_dynamic_obstacles(self):
        """동적 장애물 업데이트"""
        # 50스텝마다 새 장애물 생성
        if self.step_count % 50 == 0:
            x, y = self._get_random_pos()
            self.dynamic_obstacles.append([x, y, 30]) # 수명 30

        # 수명 감소
        for obstacle in self.dynamic_obstacles:
            obstacle[2] -= 1
            if obstacle[2] <= 0:
                self.dynamic_obstacles.remove(obstacle)
```

테스트 시나리오

선배: "이제 두 가지를 비교해봅시다."

시나리오 1: 정적 환경 (장애물 없음)

```
# config.py
ENABLE_DYNAMIC_OBSTACLES = False

python main.py
```

결과:

Policy와 Cache 일치
 → 캐시 활용 가능
 → 정적 환경에서는 캐싱 효과적!
 Success Rate: 73%

시나리오 2: 동적 환경 (장애물 생김)

```
# config.py
ENABLE_DYNAMIC_OBSTACLES = True

python main.py
```

결과:

Step 1-50: 잘 작동함
 Step 51: 새로운 장애물 등장! (오렌지색)

만약 캐시만 믿었다면?

- 옛날 경로를 그대로 따라감
- 새 장애물에 충돌! ❌

하지만 Policy 우선이므로:

- 실시간으로 장애물 감지
- 우회 경로 선택
- 안전하게 회피! ✅

Success Rate: 65% (캐시만: 30%)

💡 메타코딩의 깨달음

아하! 이제 이해했어!

캐싱의 목적:

- 정적 환경에서 빠른 실행
- 같은 맵을 반복할 때 효율적

캐싱의 한계:

- 동적 환경에서는 위험
- Policy가 최종 판단해야 함

Policy > Cache:

- 캐시는 도구일 뿐
- Policy가 최종 결정권

비유:

정적 환경 = 매일 같은 출근길
 → 네비 저장된 경로 (빠름)

동적 환경 = 공사 많은 도로
 → 눈으로 직접 확인 (안전)

메타코딩: "V5는 속도 최적화를 위한 캐싱이 목적이고, 동적 장애물은 캐싱의 한계를 보여주는 거네요!"

선배: "정확해요! 이제 완벽하게 이해했네요!"

🎮 최종 테스트!

main.py 수정:

```
# 캐시 활성화
USE_CACHE = True

while running:
    # 1. Policy 판단
    action = agent.select_action(state)

    # 2. 행동 실행
    reward, done = car.move(action, env)

    # 3. 성공한 경로만 캐시에 저장
    if USE_CACHE:
        success = env.is_goal(car.x, car.y) or (not done)
        agent.update_cache(state, action, success)

    # 4. 동적 장애물 테스트 (선택적)
    if ENABLE_DYNAMIC_OBSTACLES:
        env.update_dynamic_obstacles()
```

결과:

정적 환경 (장애물 없음):

- Policy와 Cache 일치
- 캐시 활용 가능
- 실전에서는 12ms로 빠른 실행 가능!
- Success Rate: 73%

동적 환경 (장애물 생김):

- Policy와 Cache 불일치 감지
- Policy 우선 (안전)
- 새 장애물 회피 성공
- Success Rate: 65%

📊 Cache 통계

화면에 통계가 표시되었습니다.

📦 Cache: Size=350 | Hits=142 | Agreements=89 | Conflicts=8

정적 환경:

→ Agreements 많음 (89번)

→ 캐시 활용 효과적

동적 환경:

→ Conflicts 발생 (8번)

→ Policy가 안전하게 대응

최종 보고

메타코딩은 자신감 있게 팀장님을 찾아갔습니다.

메타코딩: "추론 속도를 최적화했습니다!"

팀장님: "오! 어떻게 했어요?"

메타코딩: "캐싱 시스템을 추가했습니다. 정적 환경에서는 캐시로 빠른 실행, 동적 환경에서는 Policy로 안전성 보장합니다!"

팀장님이 여러 시나리오를 테스트했습니다.

테스트 1: 정적 환경 (같은 맵 반복)

 : "캐시를 활용하면 빠르게 실행 가능!"

Result:  빠르고 효율적

실전에서는 3.75배 빠름 가능

테스트 2: 동적 환경 (장애물 생김)

 : "어? 새로운 장애물! (Policy 감지)"

 : "우회해야겠다 (Policy 우선)"

Result:  안전하게 회피

테스트 3: 완전히 새로운 맵

 : "처음 보는 맵이네 (Cache 비어있음)"

 : "Policy로 판단하자 (학습한 능력 활용)"

Result:  성공적으로 도달

팀장님: "좋아요! 정적 환경에서는 빠르고, 동적 환경에서도 안전하네요! 이제 제품화할 수 있겠어요!"

 프로젝트 성공!

5주간의 여정이 끝났습니다.

최종 성능:

- 새로운 맵 성공률: 73%

- 학습 속도: 25% 향상 (v4)
- 추론 속도: 3.75배 빠름 가능 (v5, 정적 환경)
- 동적 환경 안전성: Policy > Cache 원칙으로 보장
- 현장 적용 가능!

에필로그: 배운 것들

메타코딩의 개발 일기 정리

5주간의 여정을 돌아보며 메타코딩은 노트를 정리했습니다.

1화에서 배운 것: 과적합의 함정

문제:

한 집(맵)에서만 청소하는 법을 배움
→ 다른 집에서는 못함

해결:

여러 집(맵)에서 청소하는 법을 배움
→ 어디서든 청소 가능!

실생활 비유:

- ❌ 집에서 카레만 1000번 만들
→ 집 카레는 완벽, 외식 카레는 못 만들
- ✅ 여러 요리를 다양하게 만들어봄
→ 어떤 요리든 만들 수 있음

핵심 교훈: 다양한 경험이 중요하다!

2화에서 배운 것: 다양성의 힘

5가지 개선:

1. 랜덤 방향 시작

Before: 항상 위쪽만 봄
After: 모든 방향을 봄

비유: 집에 갈 때 항상 같은 길 vs 여러 길 시도

2. 여러 맵 학습

Before: 맵 1개

After: 맵 20개

비유: 한 문제집 vs 여러 문제집

3. 느린 탐험 감소

Before: 빨리 포기 ($\epsilon_{\text{decay}} = 0.995$)

After: 천천히 탐험 ($\epsilon_{\text{decay}} = 0.998$)

비유:

- Before: 10번 시도 → 포기

- After: 100번 시도 → 포기

4. 경로 다양성 보상

새로운 길: +0.2 보상

같은 길 반복: -0.1 패널티

비유: 새로운 경험을 하면 보너스!

5. 학습 안정화

Gradient Clipping: 학습량 조절

비유: 폭식하지 않고 꾸준히 먹기

핵심 교훈: 다양하게 시도하고, 천천히 학습하라!

🎯 3화에서 배운 것: 진짜 실력 평가

문제:

학습한 맵으로 테스트

→ 90% 성공 (과대평가!)

해결:

새로운 맵으로 테스트
→ 68% 성공 (진짜 실력!)

비유:

- ❌ 연습문제로 시험 평가
→ 100점 (하지만 외운 것)
- ✅ 새로운 문제로 시험 평가
→ 70점 (진짜 실력)

훈련셋 vs 테스트셋:

- 훈련셋 (Training Set):
- 공부하는 문제
 - 이걸로 배움
- 테스트셋 (Test Set):
- 시험 문제
 - 이걸로 평가

핵심 교훈: 평가는 새로운 데이터로!

🎯 4화에서 배운 것: 학습 속도 개선

Learning Rate Scheduling:

- 처음 (0-1000): 빠르게! 🏃
- Learning Rate = 0.001
 - 전체 패턴 빠르게 파악
- 중간 (1000-2000): 안정적으로! 🧑
- Learning Rate = 0.0005
 - 꾸준히 학습
- 후반 (2000-3000): 세밀하게! 🐌
- Learning Rate = 0.0001
 - 미세 조정

효과:

v3: 2000 에피소드에서 목표 달성
v4: 1500 에피소드에서 목표 달성

→ 25% 빠름!

비유:

책 읽기:

- 1단계: 빠르게 훑어보기 (큰 그림)
- 2단계: 꼼꼼히 읽기 (이해)
- 3단계: 중요한 부분 정독 (완벽)

핵심 교훈: 처음엔 빠르게, 나중엔 천천히!

🎯 5화에서 배운 것: 추론 속도 최적화

목표:

같은 맵에서 반복 실행 시:

- 매번 Policy 계산 (느림)
- 캐싱으로 빠르게 실행!

해결: 캐싱 시스템 + Policy > Cache 원칙

Action Cache (캐시):

- 성공한 경로 저장
- 빠른 실행 (12ms)
- 정적 환경에서 효과적

Policy Network (정책):

- 실시간 환경 관찰
- 최종 결정권
- 동적 환경에서 필수

비유:

Cache = 네비게이션 📱

- 자주 가는 길 저장
- 빠른 경로 안내

Policy = 운전자 🚗

- 눈으로 직접 확인
- 최종 판단

정적 환경 (매일 같은 출근길):

네비: "저장된 경로로 가세요" (빠름)

→ Cache 활용! 

동적 환경 (공사 중인 도로):

네비: "저장된 경로로..." (옛날 정보)

운전자: "공사중! 우회!" (현재 상태)

→ Policy 우선! 

효과:

정적 환경:

- 캐시 활용
- 3.75배 빠름 가능!

동적 환경:

- Policy 우선
- 안전성 보장

핵심 교훈:

- 추론 시 캐싱으로 성능 최적화!
- 환경에 따라 전략 선택!
- 정적 환경: 캐시 우선 (빠름)
- 동적 환경: Policy 우선 (안전)

전체 버전 비교

v1: 첫 시도

-  DQN 기본 구현
-  과적합 문제
-  새로운 맵에서 실패

v2: 과적합 해결

-  랜덤 초기화
-  여러 맵 학습
-  경로 다양성
-  새로운 맵에서도 작동

v3: 진짜 평가

-  테스트셋 분리
-  진짜 실력 측정
-  신뢰할 수 있는 결과

v4: 속도 개선

- ✓ Learning Rate Scheduling
- ✓ 25% 빠른 학습
- ✓ 더 나은 성능

v5: 현실 대응

- ✓ 동적 환경 대응
- ✓ Policy > Cache 원칙
- ✓ 현장 적용 가능

💡 메타코딩이 배운 핵심 교훈

1. 과적합은 항상 조심하라

한 가지만 잘하는 AI < 여러 상황에 대응하는 AI

2. 다양한 경험이 중요하다

같은 것 1000번 < 다른 것 100번씩

3. 평가는 공정하게

연습문제로 평가 ✗
새로운 문제로 평가 ✓

4. 빠르게 배우되, 세밀하게 마무리

처음: 대충 빠르게
나중: 꼼꼼하게

5. 실시간 판단을 최우선으로

과거 경험도 중요하지만,
현재 상황 관찰이 더 중요!

🎓 초보자를 위한 조언

메타코딩이 이제 막 AI를 시작하는 후배에게 하는 조언:

1. 완벽하지 않아도 괜찮아요

v1은 실패했지만, 그 과정에서 많이 배웠어요.
실패는 배움의 기회입니다!

2. 한 번에 하나씩

v1 → v2 → v3 → v4 → v5
단계적으로 개선해 나가면 됩니다.

3. 비유로 이해하세요

어려운 개념도 일상적인 비유로 생각하면
쉽게 이해할 수 있어요!

- 과적합 = 문제 암기
- Learning Rate = 공부 속도
- Policy > Cache = 운전자 > 네비

4. 직접 해보세요

코드를 읽기만 하지 말고,
직접 실행해보고 결과를 확인하세요!

5. 문제가 생기면 천천히

1. 문제가 뭐지?
2. 왜 그럴까?
3. 어떻게 해결하지?
4. 결과는?
5. 배운 건?

다음 단계

메타코딩의 여정은 계속됩니다!

프로젝트 성공 후 계획:

1. 실제 로봇에 적용
2. 더 복잡한 환경 테스트
3. 여러 로봇 협업
4. 실시간 학습

계속 배워야 할 것:

- Prioritized Experience Replay
- Double DQN
- Dueling DQN
- Multi-Agent RL

마지막 조언

메타코딩의 일기 마지막 페이지:

"AI는 어렵지 않아요.
그냥 문제를 하나씩 해결하다 보면,
어느새 작동하는 AI를 만들게 됩니다.

포기하지 말고,
한 단계씩 따라가다 보면,
여러분도 할 수 있습니다!

화이팅!  "

- 메타코딩

실습해보기

직접 따라해보세요!

1주차: v1 체험

```
cd simulator-v1
python train.py # 학습
python main.py  # 테스트
```

```
# 다른 맵으로 바꿔보세요!
# → 실패하는 것을 관찰
```

2주차: v2 체험

```
cd simulator-v2
python train.py # 여러 맵에서 학습
python main.py  # 새로운 맵에서 테스트

# → 성공하는 것을 관찰!
```

3주차: v3-v5 체험

```
# 각 버전의 차이를 직접 확인해보세요!
cd simulator-v3
cd simulator-v4
cd simulator-v5
```



용어 정리 (쉬운 버전)

강화학습 (Reinforcement Learning)

AI가 시행착오를 통해 배우는 것
비유: 게임을 반복해서 하면서 늘는 것

과적합 (Overfitting)

한 가지만 잘하고 새로운 것은 못하는 것
비유: 문제를 외워서 시험 잘 봤지만 실력은 없는 것

일반화 (Generalization)

새로운 상황에서도 잘하는 것
비유: 어떤 문제가 나와도 풀 수 있는 실력

DQN (Deep Q-Network)

AI가 어떤 행동을 할지 결정하는 신경망
비유: AI의 뇌

Epsilon (엡실론)

새로운 것을 시도할 확률

높으면: 모험가 🎲

낮으면: 보수적 🎯

Learning Rate (학습률)

학습 속도

크면: 빠르지만 대충

작으면: 느리지만 정확

Policy (정책)

AI의 행동 규칙

비유: 운전자의 판단

Cache (캐시)

과거 경험 저장

비유: 메모장, 네비게이션

✅ 체크리스트

이 책을 다 읽었다면 확인해보세요!

- ☐ 과적합이 무엇인지 알겠다
- ☐ 왜 여러 맵에서 학습해야 하는지 이해했다
- ☐ 훈련셋과 테스트셋을 왜 분리하는지 알겠다
- ☐ Learning Rate Scheduling이 왜 빠른지 이해했다
- ☐ Policy > Cache 원칙을 이해했다
- ☐ 각 버전을 직접 실행해봤다
- ☐ 코드를 읽고 이해할 수 있다
- ☐ 직접 수정해볼 자신감이 생겼다

모두 체크했나요? 축하합니다! 🎉 이제 여러분도 강화학습 개발자입니다!

끝.

"메타코딩과 함께한 5주간의 여정, 고생하셨습니다!"